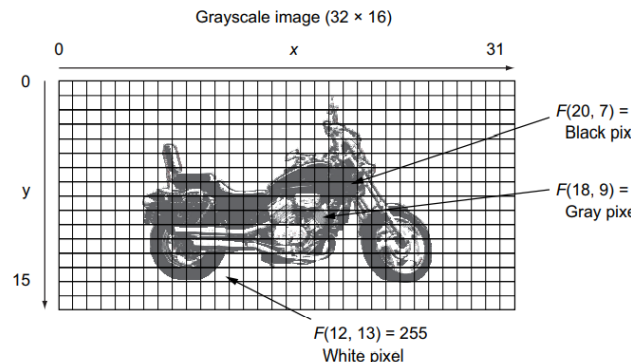


Data Preprocessing, Feature Engineering and Augmentation

Image Data

Image Representation

- A digital image is made of a grid of pixels, pixel being the raw building block of an image.
- Every image consists of a set of **pixels** in which **their values represent the intensity of light** that appears in a given place in the image.
- An image can be represented as a function of two variables x and y , which defines a two dimensional area: $F(x, y) = \langle \text{pixel_value} \rangle$.
- Note that the image coordinate system is similar to the Cartesian coordinate system: images are two-dimensional and lie on the x - y plane, with origin (0, 0) at the top left of the image.



- The pixels in an image can be stored as a binary image, a grayscale image or a color image.

Binary Image 1 bit per pixel

- It is the simplest type of image, also called monochrome image.
- Each pixel is stored as a single binary digit/bit, either 0 (black) or 1 (white).
- Binary images are mostly used for general shape or outline.
- Binary images are generated using threshold operation; if a pixel is above the threshold value, it is marked 1 and otherwise marked 0.
- $F(x, y) = [0 \text{ or } 1]$



Original Image

Binary Image

Each pixel is stored using 1 bit
Since 1 bit can represent only 2 values (0 or 1)

Gray Scale Image

- In a grayscale image, each pixel contains a value that represents the intensity of light on that specific pixel.
- The pixel values range from 0 to 255.
- Since the pixel value represents the intensity of light, the value 0 represents very dark pixels (black), 255 is very bright (white), and the values in between represent the intensity on the grayscale.
- $F(x, y) = [0 \text{ to } 255]$

Each pixel is stored using 8 bits (1 byte)
So each pixel can have 256 values (0–255)



Grayscale Image

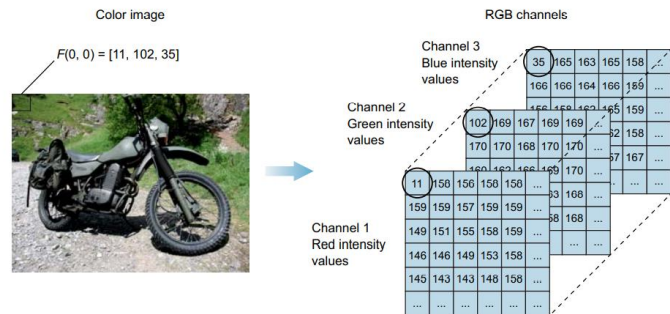
08	02	22	97	38	15	00	40	00	75	04	05	07	78	52	12	50	77	91	08
49	49	99	40	17	81	18	57	60	87	17	40	98	43	69	46	04	56	62	00
81	49	31	73	55	79	14	29	93	71	40	67	53	99	30	03	49	13	36	65
52	90	95	23	04	60	11	42	69	24	48	56	01	32	54	71	37	02	34	91
22	31	14	71	51	67	43	59	41	92	34	54	22	40	40	28	44	33	13	80
24	47	32	60	99	03	45	02	44	75	33	53	78	36	64	20	35	09	12	80
32	98	81	28	64	23	67	10	26	38	40	67	59	54	70	66	18	38	64	70
47	24	20	68	02	62	12	20	95	63	94	39	63	04	49	91	44	49	94	21
24	55	58	05	66	73	99	26	97	17	78	78	94	83	14	88	34	89	63	72
21	36	23	09	75	00	74	44	20	45	35	14	00	41	33	97	34	31	33	95
78	17	53	28	22	75	31	67	15	94	03	80	04	42	16	14	09	53	56	92
16	39	05	42	96	35	31	47	55	58	88	24	00	17	54	24	34	29	85	57
84	56	00	48	35	71	89	07	05	44	44	37	44	40	21	58	51	54	17	58
19	80	61	68	05	94	47	49	28	73	92	13	86	52	17	77	04	09	55	40
04	52	08	83	97	35	99	14	07	97	57	32	16	26	26	79	33	27	98	44
04	36	68	81	57	62	20	72	03	16	33	67	46	55	12	32	43	93	53	69
04	42	14	73	38	25	39	11	24	94	72	18	06	46	29	32	40	62	74	36
20	49	34	41	72	30	23	88	34	62	99	69	82	47	59	85	74	04	34	24
20	23	35	29	78	31	90	01	74	31	49	71	48	86	81	14	23	57	05	54
01	70	54	71	83	51	54	49	16	92	33	48	61	43	51	01	89	19	67	48

Grayscale Pixel Values

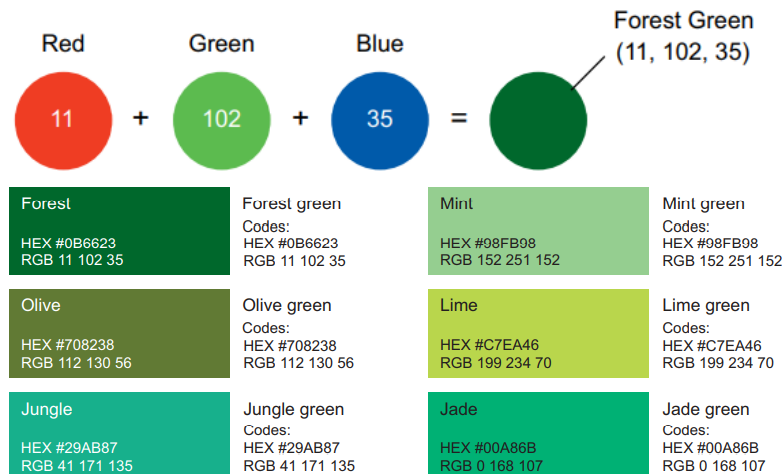
Color Image

- In color images, the pixel value is represented by three numbers representing the intensity of each color in the pixel.
- In an RGB system, for example, the value of the pixel is represented by the intensity of red, intensity of green, and intensity of blue: $F(x, y) = [\text{red (0 to 255), green (0 to 255), blue (0 to 255)}]$
- There are other color systems for images like HSV (hue, saturation, value) and Lab.
- So the standard RGB system, color images have three channels (red, green, and blue).
- They are represented by three matrices: one represents the intensity of red in the pixel, one represents green, and one represents blue.

Each channel uses 8 bits (0–255)
So each pixel uses:
 $3 \times 8 = 24$ bits (3 bytes)



- Different shades of green means different intensities of the three image colors (red, green, blue)

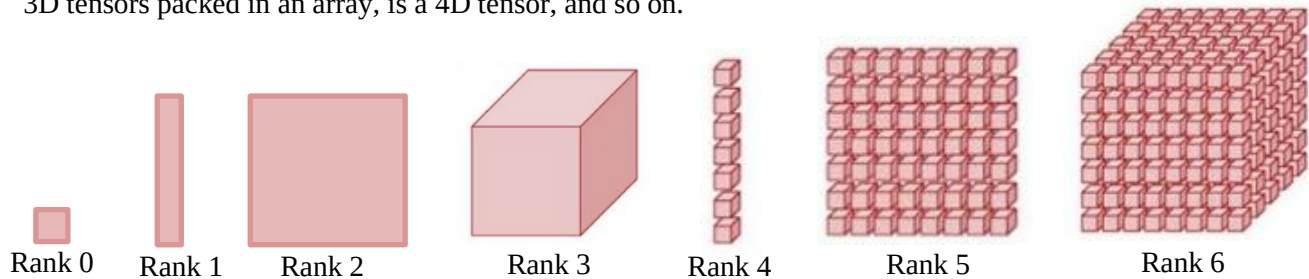


Tensor

- Tensor is a mathematical object that generalize scalars, vectors and matrices to higher dimensions.
- It is a container of data, which helps to store different dimensions of data in Neural Networks.
- Like vectors and matrices, tensors can be represented in Python using the N-dimensional array (ndarray).

Key Attributes of Tensors

- Rank:** The number of axes is called a rank of the tensor.
 - A tensor with rank 0 is a scalar.
 - A tensor with rank 1 is a vector.
 - A tensor with rank 2 is a matrix.
 - Matrices packed in a new array, is a 3-D tensor.
 - 3D tensors packed in an array, is a 4D tensor, and so on.



- Shape:** This is a tuple of integers that describes how many dimensions the tensor has along each axis.
 - For example a square matrix may have (2, 2) dimensions; a tensor of rank 3 may have (3, 5, 8) dimensions.
- Data Type (dtype):** This is the type of data contained in the tensor.
 - For example, float32, float64, uint8, int32, int64.

Image Preprocessing

1. Resizing images

- Resize images to unified dimensions (identical width and height) so they can be processed in batches.
- Care should be taken to preserve the aspect ratio, using padding or cropping when necessary, to avoid distortion.
- Models generally learn better when the aspect ratio is preserved.

2. Converting images to grayscale (optional)

- Grayscale images reduce computational complexity and memory usage.
- For some tasks, grayscale information is sufficient (e.g., detecting the presence of a motorcycle).
- However, for other applications, color is crucial, such as skin cancer detection, where color variations (e.g., red rashes) are important.

3. Converting images to tensors

- Images (PNG, JPG, BMP, etc.) are converted into numerical tensors representing pixel intensities.
- These tensors are not directly human-readable but are suitable for machine learning models.

4. Scaling pixel values

- Simple normalization (divide by 255)**
 - Formula: $x_{new} = \frac{x}{255}$
 - Keeps values in [0,1].
 - Easy and effective.
 - Preferred for simple and small models, when training a CNN from scratch.
- Mean-standard deviation normalization (z-score)**
 - Formula: $x_{new} = \frac{x - \mu}{\sigma}$, where μ is the mean and σ is the standard deviation.
 - Centers data around zero.
 - Preferred for large-scale deep learning, works well when training is unstable.
 - Pretrained models often expect this distribution.

- **Scaling to [-1, 1]**
 - Formula: $x_{new} = 2\left(\frac{x}{255}\right) - 1$
 - Preferred for GANs, models with tanh activations, and some autoencoders.

Image Data augmentation

- Data augmentation involves creating modified versions of existing images to artificially enlarge the dataset.
- Common transformations include scaling, rotations, horizontal flipping, and other affine transformations. These expose the neural network to a wide variety of image variations.
- Augmentation helps the model recognize objects in different forms, orientations, and scales, improving generalization and reducing overfitting.
- Deep learning techniques like GANs (Generative Adversarial Networks) and Variational Autoencoders can also be used to generate synthetic images.
- Additional preprocessing adjustments may include removing background colors to reduce noise, brightening or darkening images, color jittering, contrast adjustments, or other modifications depending on the dataset and task.
- Always select augmentation techniques carefully based on the dataset characteristics and the specific problem being solved.
- **Example 1**
 - Augmenting cat images by horizontal flips and scaling/zooming.



Original Image



Horizontally Flipped Image

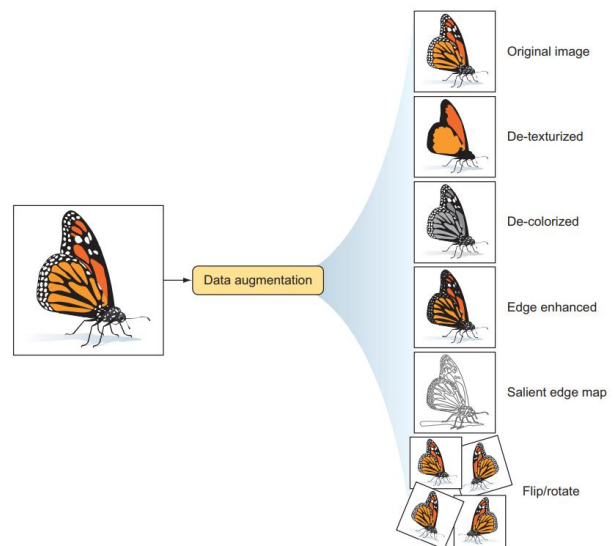


Scaled Image



Color Shifting

- **Example 2**
 - Image augmentation applied to a butterfly image.



Python Libraries for Image Data Preprocessing and Augmentation

- OpenCV (cv2): For image reading, resizing, filtering, and geometric transformations.
- Pillow (PIL): Simple image operations such as opening, saving, rotating, and cropping images.
- NumPy: Converting images to arrays and performing numerical operations.
- Keras / TensorFlow (tf.keras.preprocessing.image): Utilities for image preprocessing, data augmentation, and generating batches for training.

It was developed with a focus on enabling fast experimentation.

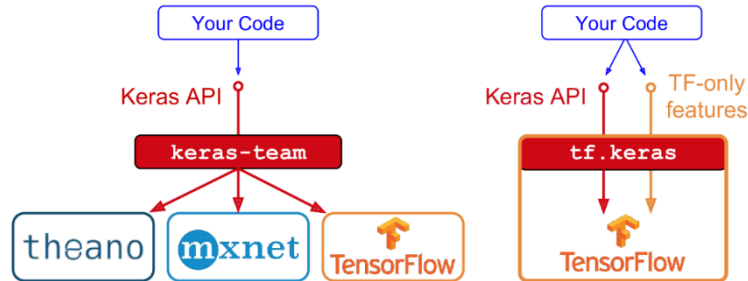
To perform the heavy computations required by neural networks, keras relies on a computation backend.

Three popular such deep learning libraries are TensorFlow, Microsoft Cognitive Toolkit (CNTK) and Theano.

New releases of Keras can also be run on Apache MXNet, Apple's Core ML, Javascript or Typescript (to run Keras code in a web browser), or PlaidML (which can run on all sorts of GPU devices, not just Nvidia).

TensorFlow itself now comes bundled with its own Keras implementation called tf.keras.

It only supports TensorFlow as the backend, but it has the advantage of offering some very useful extra features.



- TorchVision (PyTorch): Provides transforms for resizing, normalization, conversion to tensors, and augmentations like flips, rotations, and color jitter.
- Albumentations: A fast and flexible library for advanced image augmentation.
- Imgaug: Offers a wide range of image augmentation techniques with easy-to-use pipelines.

Some Commonly Used Image Datasets

- **MNIST:** Handwritten digits, grayscale 28×28 images, containing a training set of 60,000 examples, and a test set of 10,000 examples.
- **Fashion-MNIST:** 28×28 grayscale images of clothing items.
- **CIFAR-10 / CIFAR-100:** Small color 32×32 images in 10 or 100 classes, containing a training set of 50,000 training images and 10,000 test images.
- **ImageNet:** Large-scale, high quality dataset with around 14 million of labeled/hand-annotated images across almost 22,000 separate object classes; created and maintained by Princeton and Stanford University.
- **COCO:** Dataset for object detection, segmentation, and captioning.
- **CelebA:** Large dataset of celebrity faces with attribute annotations